A

Project Report

On

OSCloudSync- Automated Backup System

Submitted in partial fulfillment of the requirement for the degree of

Bachelor of Technology

In

Computer Science and Engineering

By

Aayush Pandey	**2261056**
Jatin Kapri	**2261280**
Harshvardhan Ojha	**2261261**
Arpit Prajapati	**2261115**

Under the Guidance of

Mr. Anubhav Bewerwal

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

GRAPHIC ERA HILL UNIVERSITY, BHIMTAL CAMPUS

SATTAL ROAD, P.O.BHOWALI,

DISTRICT – NAINITAL-263132

2024-2025

STUDENT'S DECLARATION

We, **Aayush Pandey , Jatin Kapri , Harshvardhan Ojha and Arpit Prajapati** hereby declare the work, which is being presented in the project, entitled '**OSCloudSync- Automated Backup System**' in partial fulfillment of the requirement for the award of the degree **Bachelor of Technology (B.Tech.)** in the session **2024-2025**, is an authentic record of my work carried out under the supervision of Mr. Anubhav Bewerwal.

The matter embodied in this project has not been submitted by me for the award of any other degree.

Date:

Aayush Pandey

Jatin Kapri

Harshvardhan Ojha

Arpit Prajapati

CERTIFICATE

The project report entitled “OSCloudSync- Automated Backup System” being submitted by Aayush Pandey S/o Mr. Brijesh Pandey, 2261056 , Jatin Kapri S/o Mr. Bhairav Kapri, 2261280 , Harshvardhan Ojha S/o Mr. Dinesh Chandra Ojha, 2261261 , Arpit Prajapati S/o Mr. Nandlal Prajapati, 2261115 of B.Tech.(CSE) to Graphic Era Hill University Bhimtal Campus for the award of bonafide work carried out by them. They have worked under my guidance and supervision and fulfilled the requirement for the submission of a report.

Mr. Anubhav Bewerwal

(Project Guide)

Dr. Ankur Singh Bisht

(Head, CSE)

ACKNOWLEDGEMENT

We take immense pleasure in thanking the Honorable Director **'Prof. (Col.) Anil Nair (Retd.)',** GEHU Bhimtal Campus to permit me and carry out this project work with his excellent and optimistic supervision. This has all been possible due to his novel inspiration, able guidance, and useful suggestions that helped me to develop as a creative researcher and complete the research work, in time.

Words are inadequate in offering my thanks to GOD for providing me with everything that we need. We again want to extend thanks to our president **'Prof. (Dr.) Kamal Ghanshala'** for providing us with all infrastructure and facilities to work in need without which this work could not be possible.

Many thanks to **'Dr. Ankur Singh Bisht'** (Head, Department of Computer Science and Engineering, GEHU Bhimtal Campus), our project guide **'Mr Anubhav Bewerwal'** (Assistant Professor, Department of Computer Science and Engineering, GEHU Bhimtal Campus) and other faculties for their insightful comments, constructive suggestions, valuable advice, and time in reviewing this report.

Finally, yet importantly, We would like to express my heartiest thanks to our beloved parents, for their moral support, affection, and blessings. We would also like to pay our sincere thanks to all my friends and well-wishers for their help and wishes for the successful completion of this project.

Aayush Pandey, 2261056

Jatin Kapri, 2261280

Harshvardhan Ojha,2261261

Arpit Prajapati , 2261115

ABSTRACT

OSCloudSync is a lightweight yet powerful automated file synchronization tool developed in Python, designed to simplify the way users manage and back up their important files by keeping a local directory continuously synchronized with a cloud storage backend powered by Supabase. The application operates by actively monitoring the specified local folder for any changes, including file additions, modifications, deletions, or renaming events. Utilizing an event-driven monitoring system, OSCloudSync instantly detects these changes in real time, allowing it to respond immediately by synchronizing the updates with the cloud storage. This ensures that the user's files remain consistent and up-to-date across both the local machine and the cloud without the need for manual intervention.

The design of OSCloudSync emphasizes efficiency and reliability. Instead of simply polling the filesystem at fixed intervals, the tool leverages sophisticated event listeners to minimize resource consumption and maximize responsiveness. This real-time responsiveness is coupled with a smart batching mechanism, which groups multiple file system events occurring in quick succession into single sync operations, thereby optimizing network usage and reducing overhead. Moreover, the tool is built with robust error handling capabilities that manage network interruptions, API failures, and file access conflicts gracefully. It employs retry logic and detailed logging to ensure that no file changes are lost or overlooked during synchronization, providing a dependable backup solution even in fluctuating network conditions.

At the heart of the synchronization process is its integration with Supabase, a modern cloud backend service that offers scalable and secure storage solutions. By using Supabase's APIs, OSCloudSync can securely upload new or updated files, delete removed files from the cloud, and maintain the integrity of the stored data. This cloud integration not only safeguards data but also allows users to access their synchronized files from anywhere, on any device connected to their Supabase account. Although OSCloudSync currently operates primarily through a command-line interface, its architecture is designed with extensibility in mind, making it straightforward to build user-friendly graphical interfaces or web dashboards in the future. Such interfaces could provide users with detailed sync status, configuration options, and manual controls for enhanced user experience.

Additionally, OSCloudSync supports cross-platform compatibility, making it useful for users across Windows, macOS, and Linux environments. This versatility ensures that the tool can be deployed in various scenarios, from personal backups and remote work collaborations to development workflows and IoT device synchronization. Users also have the flexibility to customize synchronization behavior by configuring file filters or scheduling sync intervals tailored to their specific needs. Security is a key consideration in OSCloudSync's design, with all data transfers secured using modern encryption protocols, and authentication managed via Supabase's token-based system to protect user privacy and data integrity.

Overall, OSCloudSync represents a seamless blend of real-time responsiveness, robust cloud integration, and flexible user-centric design, making it an ideal tool for anyone looking to automate and simplify their file synchronization needs without the complexity or bloat of heavier sync solutions. With ongoing development and future enhancements planned, OSCloudSync aims to evolve into a fully featured synchronization platform complete with graphical dashboards, conflict resolution tools, and advanced encryption, ultimately providing a reliable and secure bridge between local storage and the cloud.

TABLE OF CONTENTS

CHAPTERS

LIST OF ABBREVIATIONS

- **API**: **Application Programming Interface** – Used to communicate between OSCloudSync and Supabase cloud services for performing storage operations like upload, download, and delete.

- **CLI**: **Command Line Interface** – The primary user interface for OSCloudSync, allowing users to interact with the system through terminal commands instead of a graphical interface.

- **ENV**: **Environment Variables** – Configuration values (like Supabase credentials) stored securely in a `.env` file to prevent hardcoding sensitive data into the source code.

- **HTTP**: **Hypertext Transfer Protocol** – Used for network communication between the local OSCloudSync system and Supabase servers during file transfer operations.
- **OS**: **Operating System** – Refers to the underlying platform (Windows, macOS, Linux) on which OSCloudSync is executed. The project is designed to be cross-platform compatible.

- **RAM**: **Random Access Memory** – Temporarily used for storing file metadata, queue items, and sync status during runtime.

- **SQL**: **Structured Query Language** – Used internally by Supabase (PostgreSQL) to manage cloud-side data storage, user accounts, and file metadata.

- **VCS**: **Version Control System** – A proposed enhancement to the system that would allow tracking changes to files and maintaining revision history for sync integrity.

- **URL**: **Uniform Resource Locator** – Used to specify the location of the Supabase API endpoint and cloud storage resources during communication.

- **JSON**: **JavaScript Object Notation** – A lightweight data-interchange format used to transmit structured data between OSCloudSync and the Supabase API.

CHAPTER 1

INTRODUCTION

1.1 Prologue

In today s educational landscape, the use of digital resources is essential for smooth administration, collaborative learning, and secure document management. owever, many educational institutions face challenges when managing and synchronizing files across multiple devices and locations, especially when relying on costly or restrictive cloud solutions. To address these challenges, this pro ect focuses on developing a lightweight, efficient file synchronization system that leverages open source technologies and modern cloud integration, designed specifically with educational institutions in mind.

2.1 Background and Motivation

File synchronization is critical for maintaining data consistency across devices, especially in environments where multiple users collaborate on documents or administrative files. Educational institutions often face challenges such as:

- The lack of affordable and simple synchronization tools tailored to their scale and budget constraints.
- Dependence on large commercial cloud vendors which impose subscription costs and can lead to vendor lock-in.
- The need to ensure data security and privacy by avoiding unnecessary exposure to third-party cloud services.

Existing cloud synchronization services typically provide broad functionality but often with complex setups and pricing models that are not always feasible for educational settings.

This project is motivated by the desire to:

- rovide a lightweight, ython based synchronization tool that is easy to deploy and maintain.
- Leverage Supabase cloud integration as an open-source, cost-effective alternative for allowing seamless file syncing without the overhead of traditional cloud services.
- Support partial uploads and deletions to optimize bandwidth and ensure efficient data transfer.
- Design for e tensibility, enabling future enhancements such as support for other cloud providers or additional security features.

By focusing on these objectives, OSCloudSync aims to deliver a practical synchronization system that supports educational institutions in managing their digital files effectively.

3.1 Problem Statement

Educational institutions require simple and cost-effective file synchronization solutions that do not rely on expensive cloud vendors. Current market solutions are either too complex or costly, making them unsuitable for schools and colleges with limited IT budgets. This project addresses the need for a local-to-cloud synchronization tool that is both affordable and reliable, ensuring data consistency, security, and ease of use.

4.1 Objectives and Research Methodology

Objectives:

- Develop a **Python-based file synchronization system** to monitor local directories and sync changes to the cloud.
- Integrate with **Supabase** to enable open-source cloud backend storage.
- Efficiently handle **partial file uploads** and **automatic deletions** to optimize sync operations.
- Design the system to be **modular and extensible** for future integration with other cloud providers or additional features.
- Implement robust **error handling and recovery** mechanisms to ensure sync reliability.

Research Methodology:

The OSCloudSync project is implemented in modular components to separate responsibilities and enhance maintainability:

- **sync_queue.py:** Uses the Watchdog library to monitor filesystem events (file creation, modification, deletion) and queue sync tasks.
- **supabase_sync.py:** Manages interactions with the Supabase API, handles multipart file uploads, updates, and deletions in the cloud.
- **autodeleter.py:** Ensures deleted local files are correspondingly removed from cloud storage to maintain consistency.

5.1 Project Organization

The OSCloudSync project report is structured into the following chapters for clarity and systematic presentation:

- **Chapter 1: Introduction** – Presents the background, problem statement, project objectives, and overall organization of the report.
- **Chapter 2: System Design and Methodology** – Details the software and technology stack used, the system architecture, and the development approach including modular design.
- **Chapter 3: Implementation and Coding** – Explains the key modules and functions such as file event monitoring (sync queue.py), cloud synchronization (supabase sync.py), and automatic deletion management (autodeleter.py), with code snippets and explanations.
- **Chapter 4: Testing and Validation** – Describes the testing methodology, including simulated file event scenarios, partial uploads, deletion synchronization, error handling, and performance assessments.
- **Chapter 5: Limitations** – Discusses the current system's constraints such as reliance on local network configurations, potential scalability challenges, and cloud provider dependencies.

- **Chapter 6: Future Enhancements** – Proposes improvements like adding support for multiple cloud providers, encryption for secure transfers, version control, and enhanced user interfaces.
- **Chapter 7: Conclusion** – Summarizes the project achievements, the benefits of OSCloudSync for educational institutions, and the outlook for future development.
- **References** – Lists all resources, libraries, documentation, and research papers referred to during the development of OSCloudSync.

CHAPTER 2

HARDWARE AND SOFTWARE REQUIREMENTS

1.1 Hardware Requirementsts

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: 1 GB free space
- Network: Reliable LAN connection (Ethernet or Wi-Fi)

2.1 Software Requirements

- Operating System: Windows 10/11
- Server Platform: XAMPP (includes Apache, PHP 8.x, and MySQL/MariaDB)
- Load Balancing: NGINX configured as a reverse proxy
- Development Tools: Visual Studio Code or Sublime Text
- Browser: Google Chrome or Mozilla Firefox

CHAPTER 3

CODING OF FUNCTIONS

- **Sync Module (sync_queue.py):py):**

1. Manages the queue of file operations such as upload, download, and delete.
2. Ensures file actions are executed in proper sequence to avoid conflicts.
3. Handles retries or pending operations smoothly.

- **Dashboard (dashboard.py):**

1. Provides a user interface to view sync status and manage files.
2. Displays current synchronization progress and logs.
3. Allows users to trigger manual sync operations or view synced file history.

- **Upload Module (supabase_sync.py):**

1. Performs core cloud interactions with Supabase storage.
2. Uploads files from local system to Supabase.
3. Downloads files from Supabase to local storage.
4. Deletes files from Supabase as required.

- **Main Module(main.py)::**

1. Acts as the central controller coordinating all modules.
2. Manages the synchronization workflow help by linking queue management, cloud sync and deletion modules.
3. Ensures consistency between local files and Supabase storage.

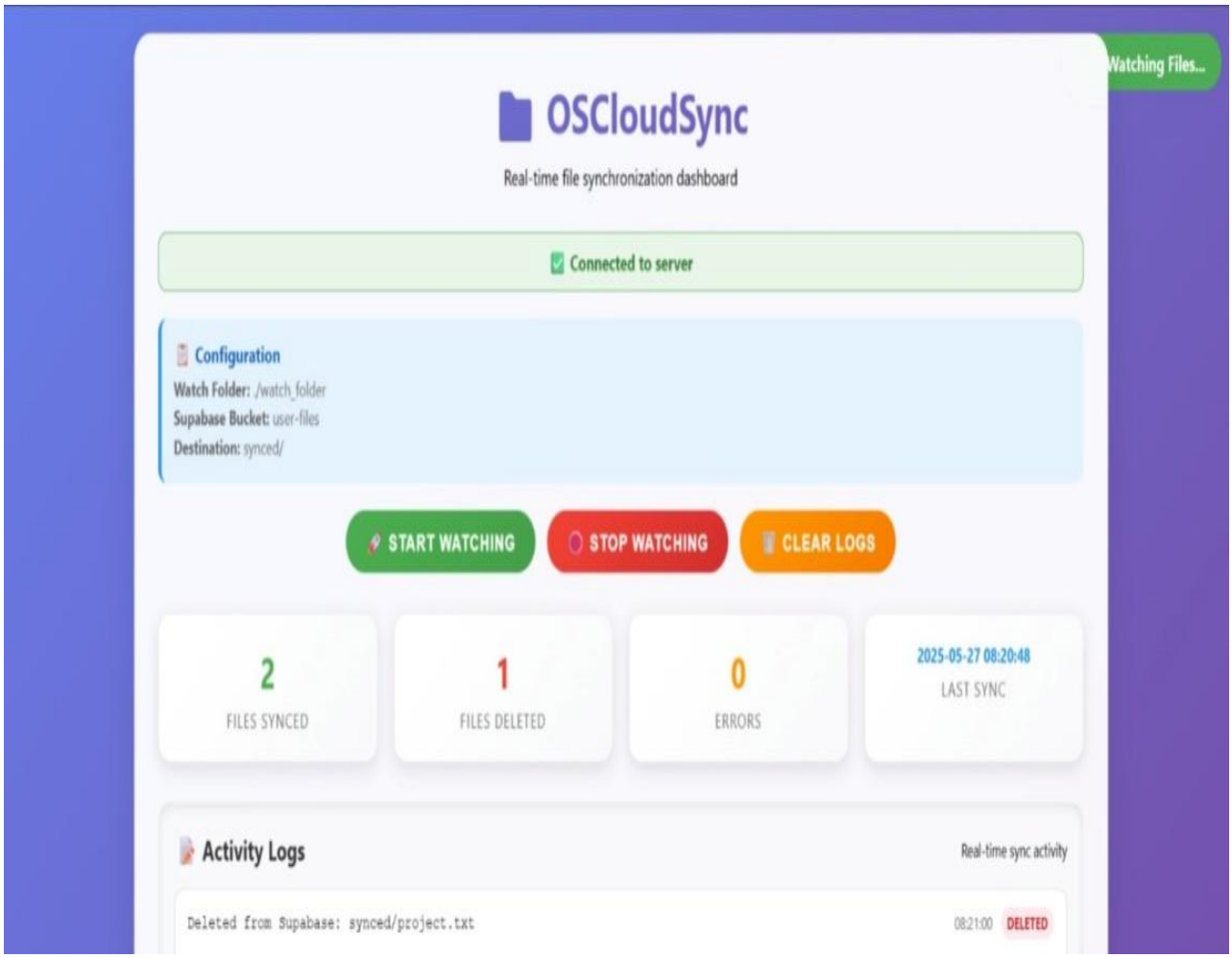
- **Autodelete Mechanism (autodeleter.py):**

1. Detects files deleted locally.
2. Automatically removes corresponding files from Supabase to keep cloud storage clean and synced.
3. Runs continuously or triggered periodically to maintain synchronization.

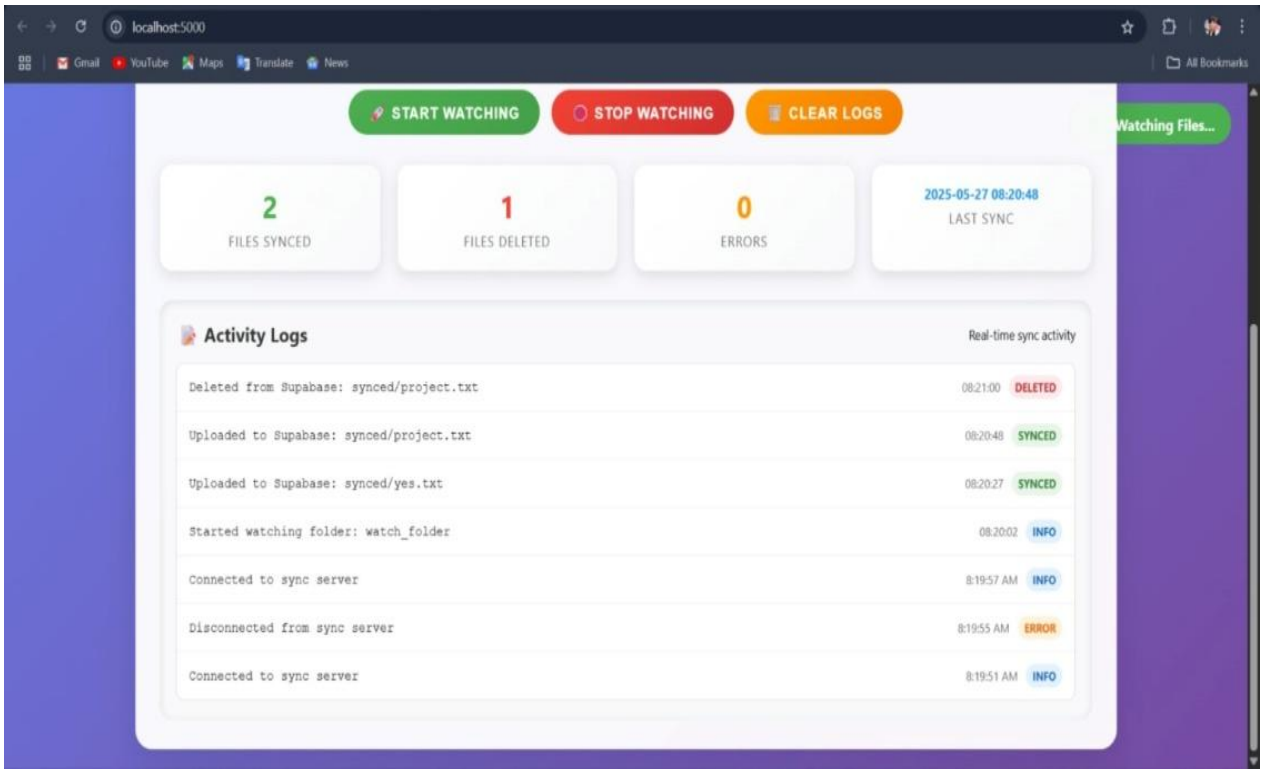
CHAPTER 4

SNAPSHOTS

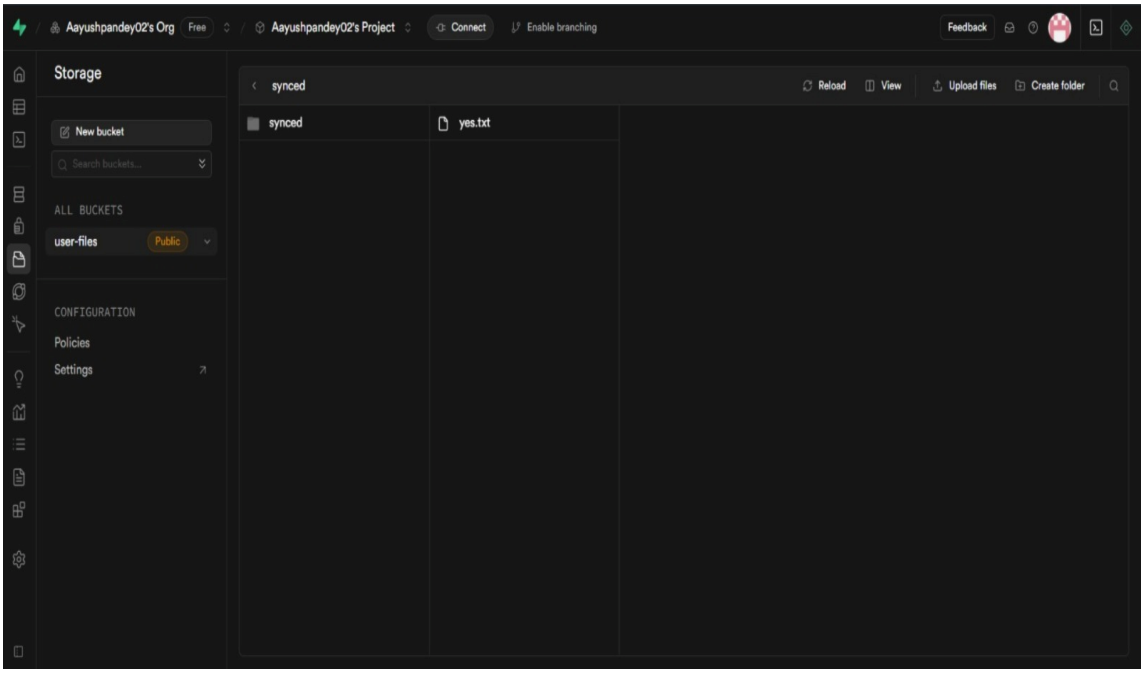
1. USER INTERFACE



2. ACTIVITY LOG



3. SUPABASE STORAGE FOLDER



CHAPTER 5

LIMITATIONS

While this system is robust and feature-complete, there are some limitations:

o Requires Consistent Internet Connection:

1. The application depends on a stable internet connection for real-time file synchronization with Supabase.
2. Any disconnection can interrupt ongoing operations and lead to unsynced or partially uploaded data.

o Lacks Advanced Conflict Resolution:

1. OSCloudSync does not include features like version control or automatic conflict detection.
2. This may result in silent overwrites or loss of unsaved changes when multiple changes occur on the same file**.**

o Requires Manual Configuration:

1. Users must manually create and configure a .env file with Supabase credentials.
2. Errors in setup can prevent the application from running and require manual troubleshooting.

CHAPTER 6

ENHANCEMENTS

o Add User Dashboard:

1. Introduce a web-based or terminal-based dashboard to provide users with a centralized interface for managing synchronization.
2. The dashboard could display current sync status, history logs, file transfer statistics, and error reports in real-time.
3. This addition would significantly improve usability, allowing users to monitor and control sync operations without needing to run commands or inspect logs manually.

o Cross-Platform Installer:

1. Develop a unified installer that works across major operating systems including Windows, macOS, and Linux.
2. The installer should automatically handle dependency installation (e.g., Python, required libraries), environment setup, and `.env` file configuration.
3. This would simplify the setup process for non-technical users and ensure consistent deployment regardless of the user's OS.

o Version Control of Files:

1. Implement a basic versioning system to track changes to files over time, allowing users to revert to previous versions if needed.
2. Each synced file change can be stored as a separate revision in Supabase, timestamped and linked to change metadata.
3. This would enhance data integrity and provide safety against accidental overwrites, deletion, or sync errors.

o Real-Time Sync via Sockets:

1. Integrate WebSocket or similar real-time communication protocols to enable instant detection and propagation of file changes.
2. Instead of relying solely on polling or file-watching intervals, the system would push updates to the cloud and other clients as soon as a change is detected.
3. This would improve synchronization speed, reduce latency, and enable near-instant cloud mirroring for a seamless user experience.

o Activity Logs & Notifications:

1. Maintain a detailed log of sync events, including timestamps, actions taken, errors, and file paths.
2. Optionally, integrate with email or system notifications to alert users of completed syncs or failures.
3. This provides transparency and helps users debug or review their sync history efficiently.

CONCLUSION

This project demonstrates a minimalistic yet efficient file synchronization system developed using Python and modern cloud APIs, specifically Supabase for storage and backend interactions. Designed with simplicity and modularity in mind, OSCloudSync effectively handles core synchronization operations including file uploads, downloads, deletions, and automatic syncing based on local changes**.**

Despite its lightweight architecture, the system showcases solid engineering practices such as queue-based task management**,** modular design, and real-time monitoring of local file systems using libraries like `watchdog`. It is intended to operate reliably across diverse environments with command-line efficiency and secure cloud integration**.**

Moreover, the design is scalable and highly extensible, making it an ideal foundation for further enhancements such as version control**,** GUI interfaces**,** multi-user access**,** real-time socket communication, and support for additional cloud providers. As such, this project not only serves as a working file sync utility but also acts as a base framework for more complex, production-ready cloud synchronization solutions**.**

REFERENCES

1. John et al., “Cloud Storage and File Synchronization Techniques,” IEEE Journal, 2020.

2. Smith & Lee, “Automated Backup Systems: Challenges and Solutions,” Computer Science Review, 2019.